

Chapter 5

Introduction to Public Key Cryptography

Stefan Dziembowski

www.crypto.edu.pl/Dziembowski

University of Warsaw



Plan



1. Public key cryptography – an overview
2. The key management problem
 1. qualified signatures
 2. public key infrastructure
3. Identity-based cryptography

Public-Key Cryptography

also called: **asymmetric cryptography**



Ralph Merkle (1974)

Whitfield Diffie and Martin Hellman (1976)

A little bit of history

Diffie and Hellman were the first to publish a paper containing the idea of the public-key cryptography:

W.Diffie and M.E.Hellman,
New directions in cryptography

IEEE Trans. Inform. Theory, IT-22, 6, 1976, pp.644-654.

A similar idea was described by **Ralph Merkle**:

in **1974** he described it in a project proposal for a Computer Security course at UC Berkeley

(it was rejected)

in **1975** he submitted it to the CACM journal (it was rejected)

(see www.merkle.com/1974/)

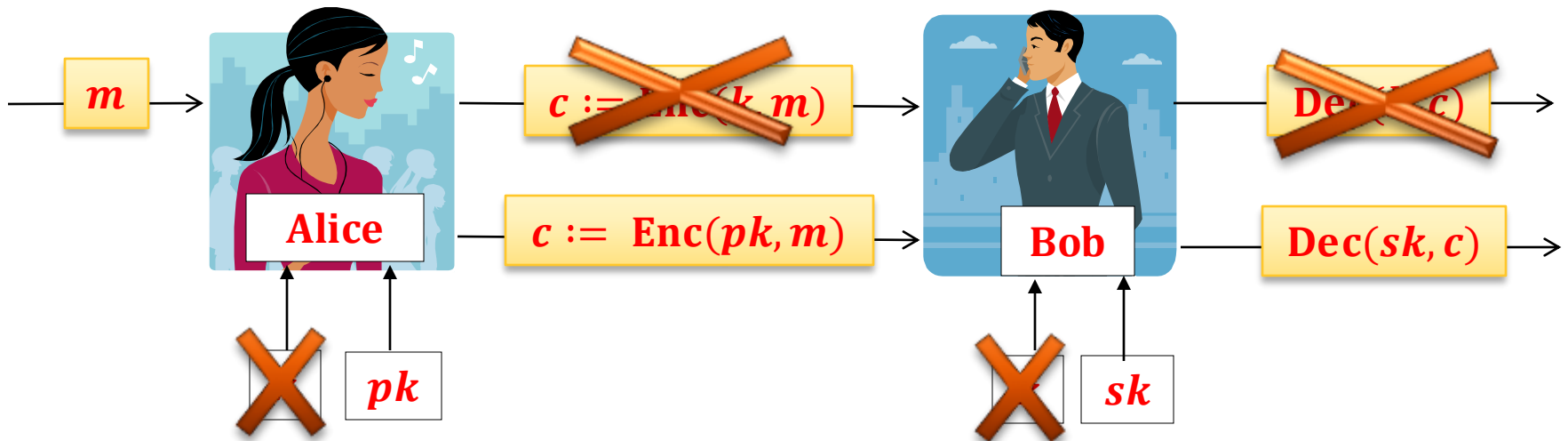
In **1997** the GCHQ (the British equivalent of the NSA) revealed that they knew it already in **1973**.

The idea

Instead of using one key k ,
use **2** keys (pk, sk), where
 pk is used for **encryption**,
 sk is used for **decryption**.

pk can be public,
and only sk has to
be kept secret!

That's why it's
called: **public-key
cryptography**

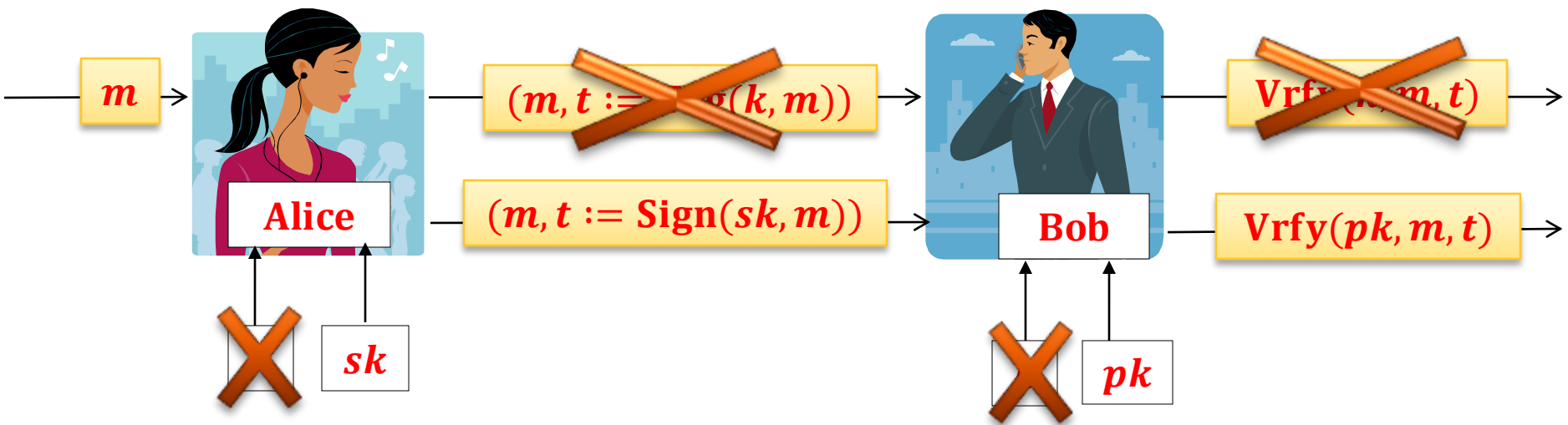


The same thing works for authentication

- ***sk*** is used for **computing a tag**,
- ***pk*** is used for **verifying correctness of the tag**.

this will be called
“**signatures**”

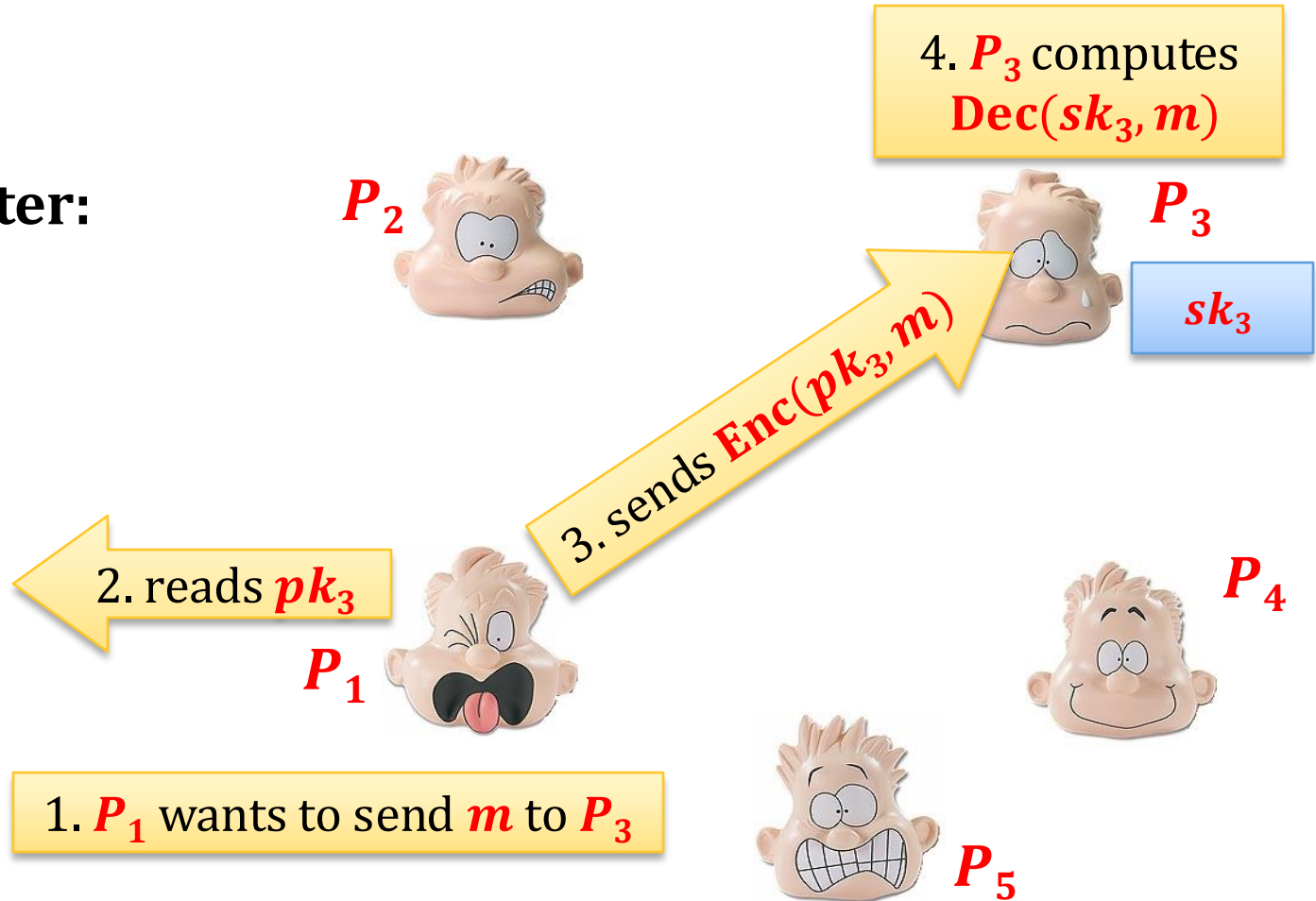
Sign – the signing algorithm



Anyone can send encrypted messages to anyone else

public register:

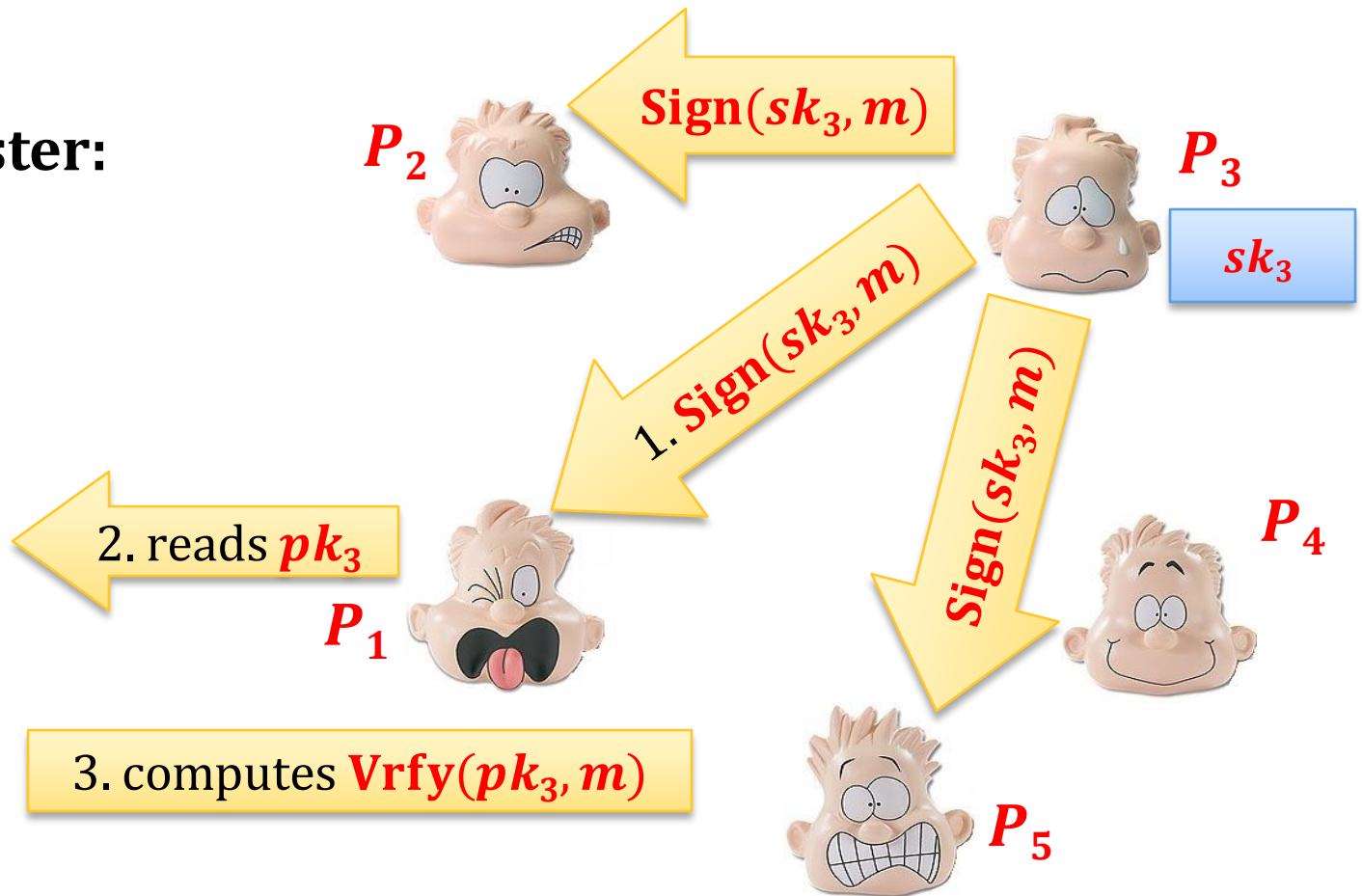
pk_1
pk_2
pk_3
pk_4
pk_5



Anyone can verify the signatures

public register:

pk_1
pk_2
pk_3
pk_4
pk_5



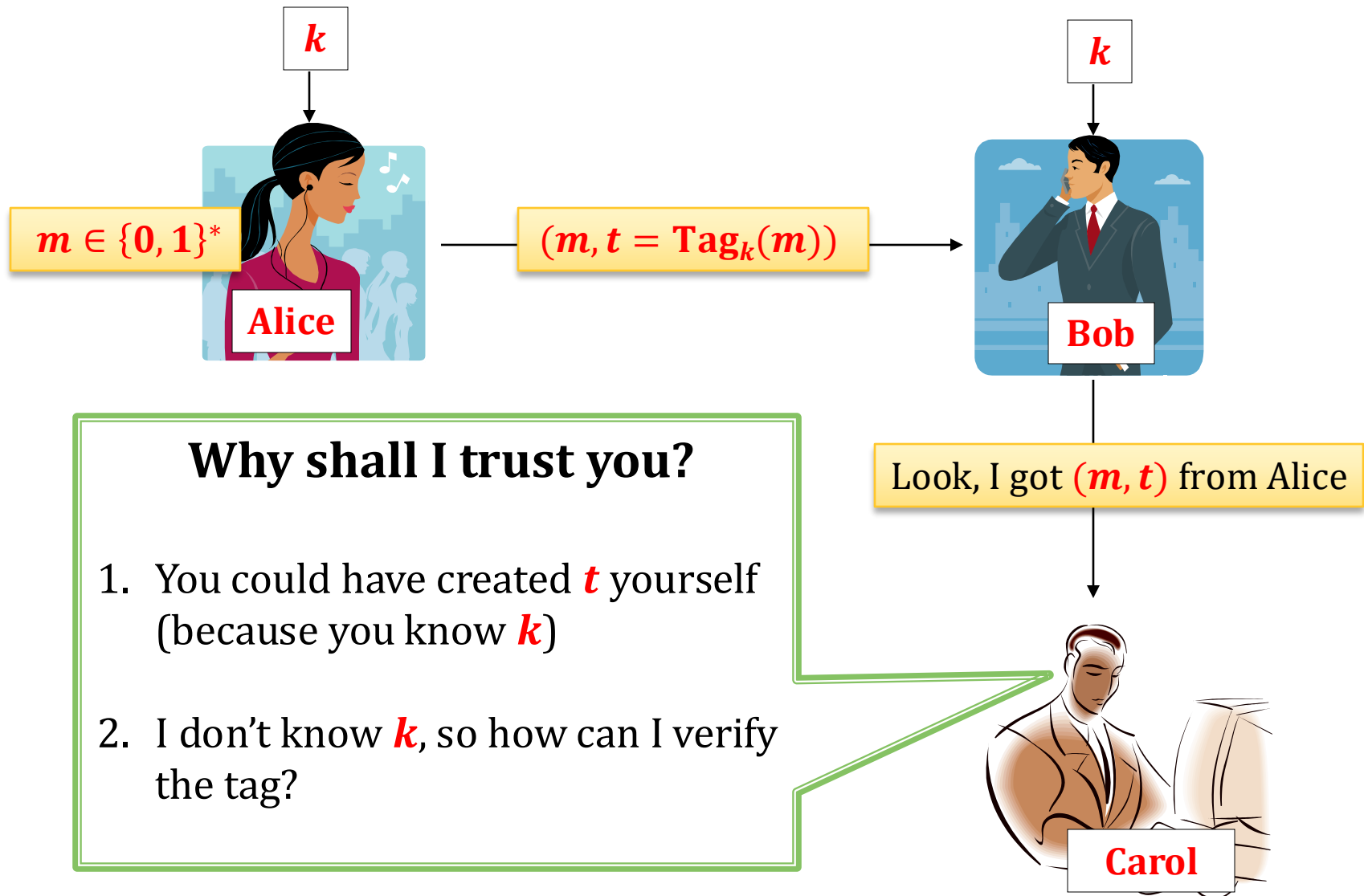
Advantages of the signature schemes

Digital signatures are:

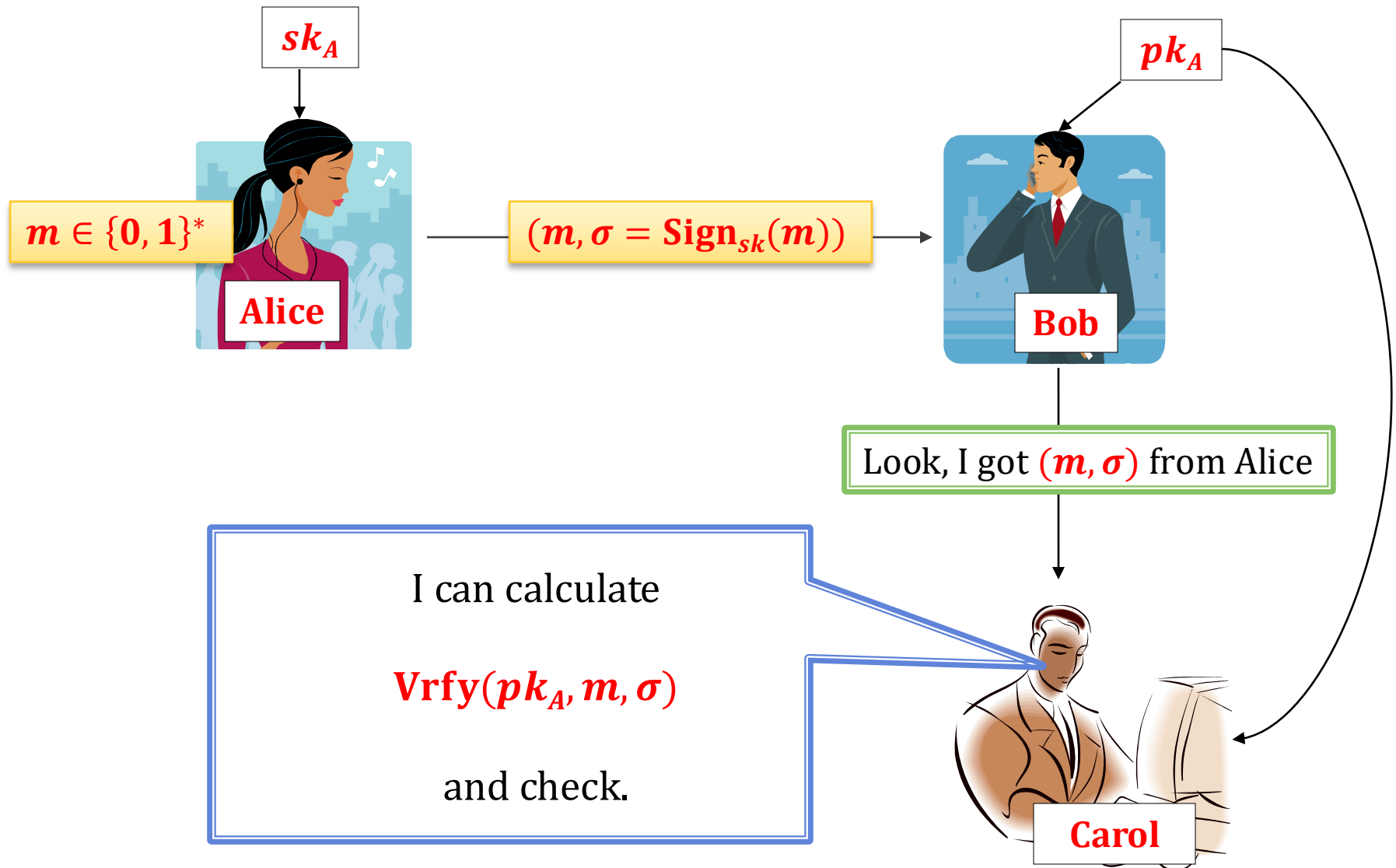
1. **publicly verifiable**,
2. **transferable**, and
3. provide **non-repudiation**

(we explain it on the next slides)

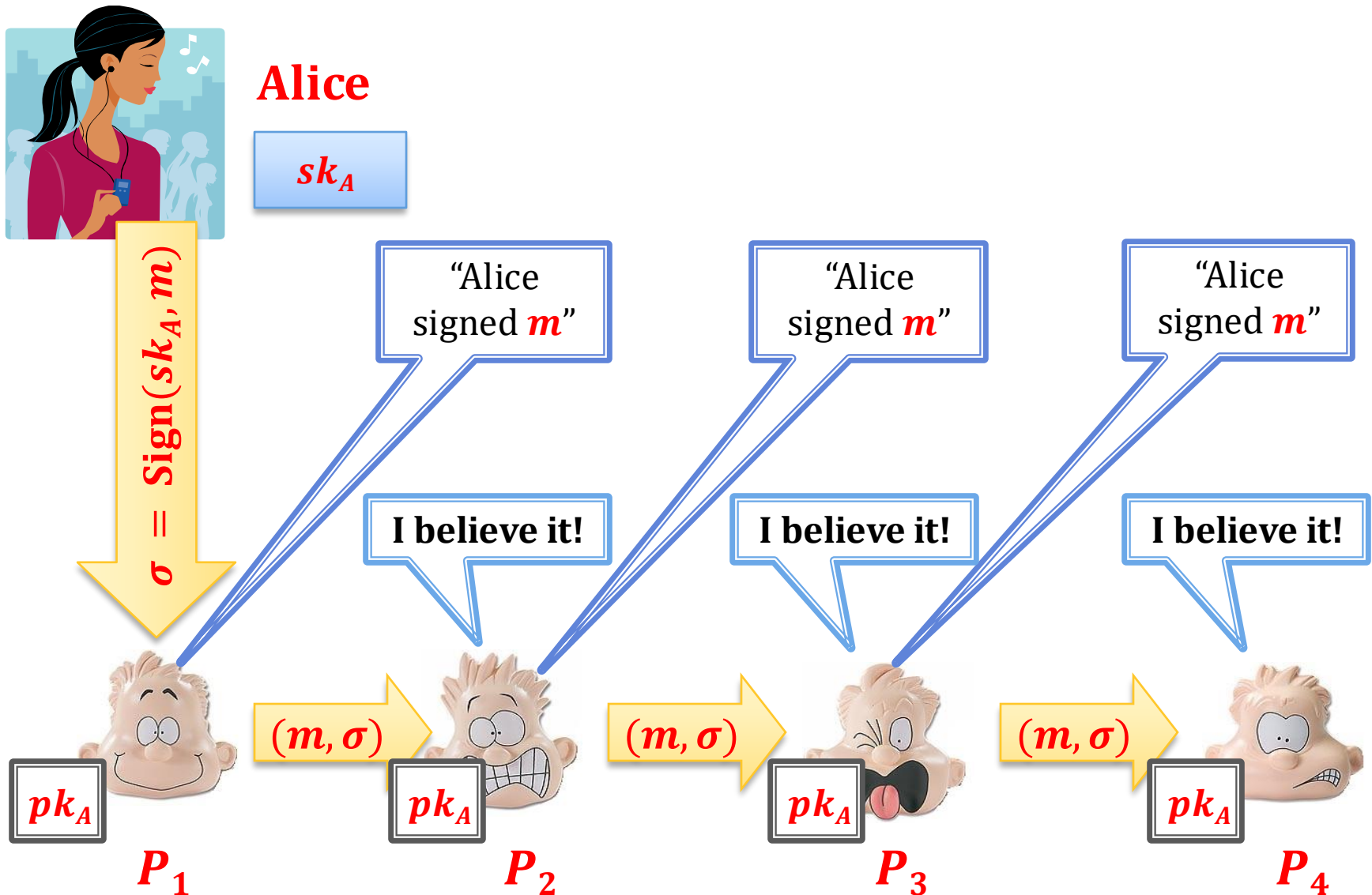
Look at the MACs...



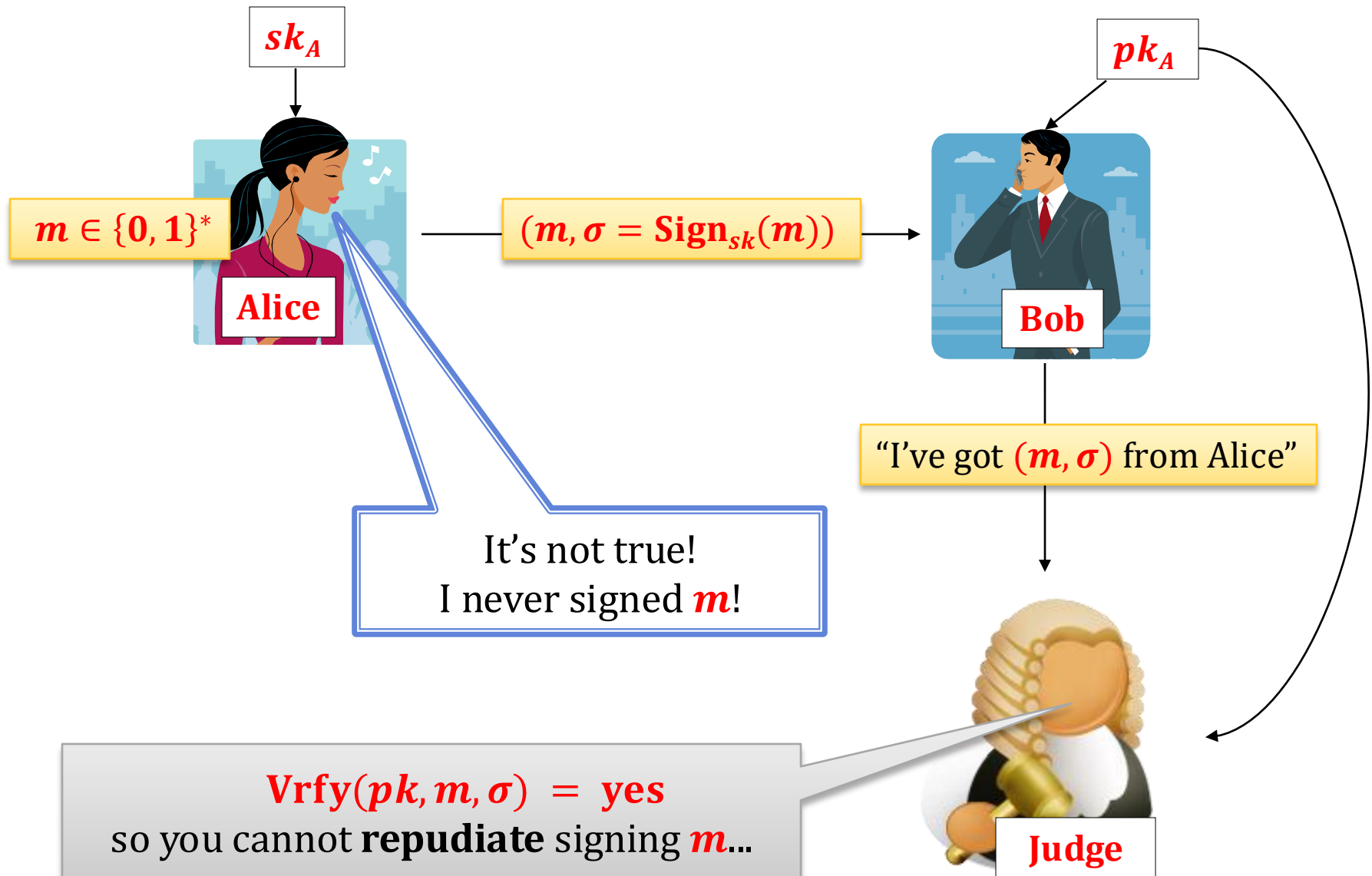
Signatures are publicly-verifiable!



So, the signatures are transferable



Non-repudiation



Things that need to be discussed

- Who **maintains “the register”**?
- How to **contact it securely**?
- How to **revoke the key** (if it is lost)?
- ...



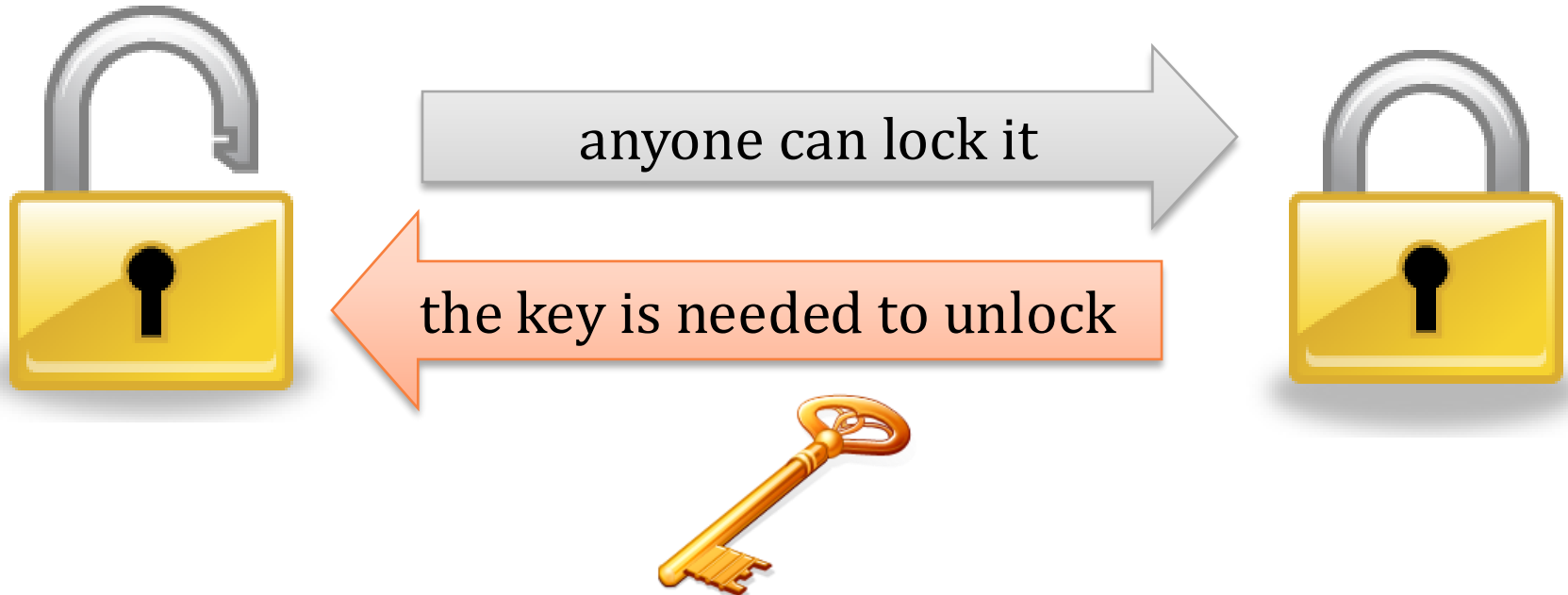
We will discuss these things
later, when we will be talking
about the
Public-Key Infrastructure

But is it possible?

In the “physical world”: **yes!**

Examples:

1. “normal” signatures
2. padlocks:



Diffie and Hellman (1976)

Diffie and Hellman proposed the public key cryptography in **1976**.

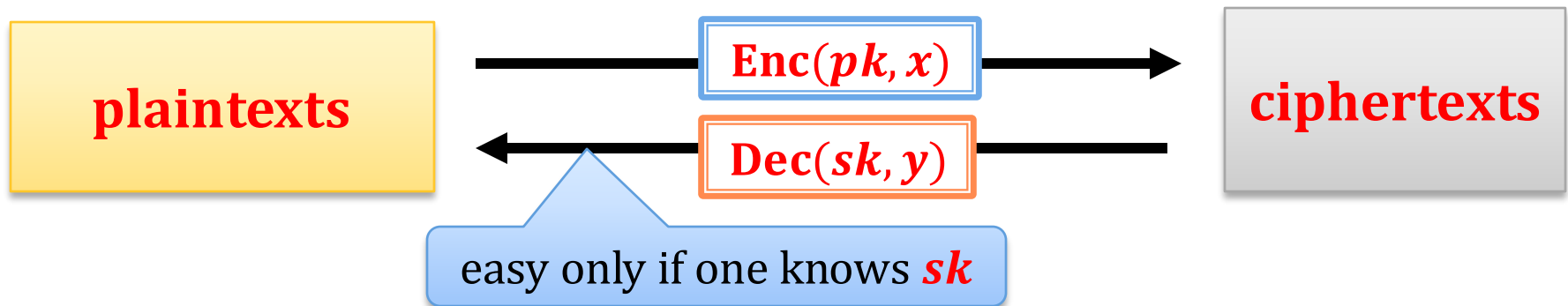
They just proposed the **concept**, not the **implementation**.

They have also shown a protocol for **key-exchange**.

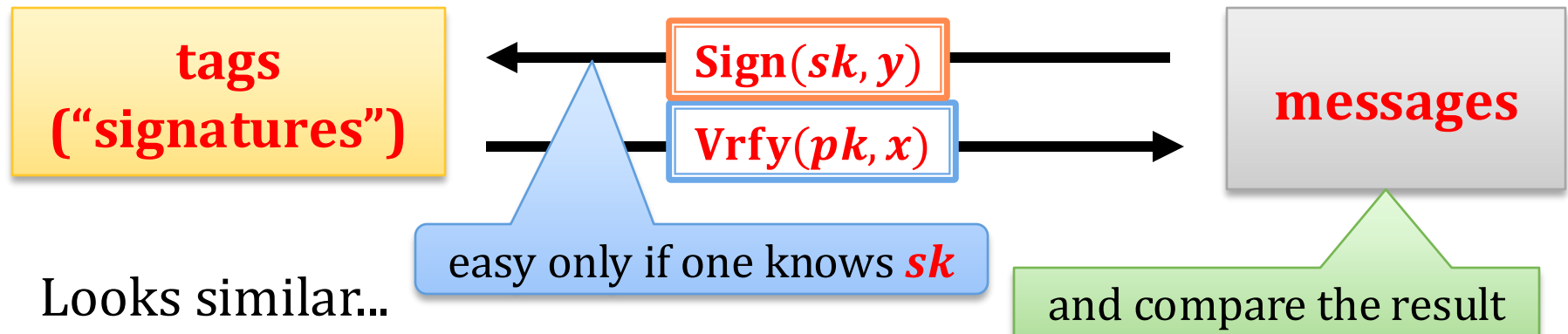
The observation of Diffie and Hellman:

(pk, sk) – the key pair

public-key encryption:



signature schemes:



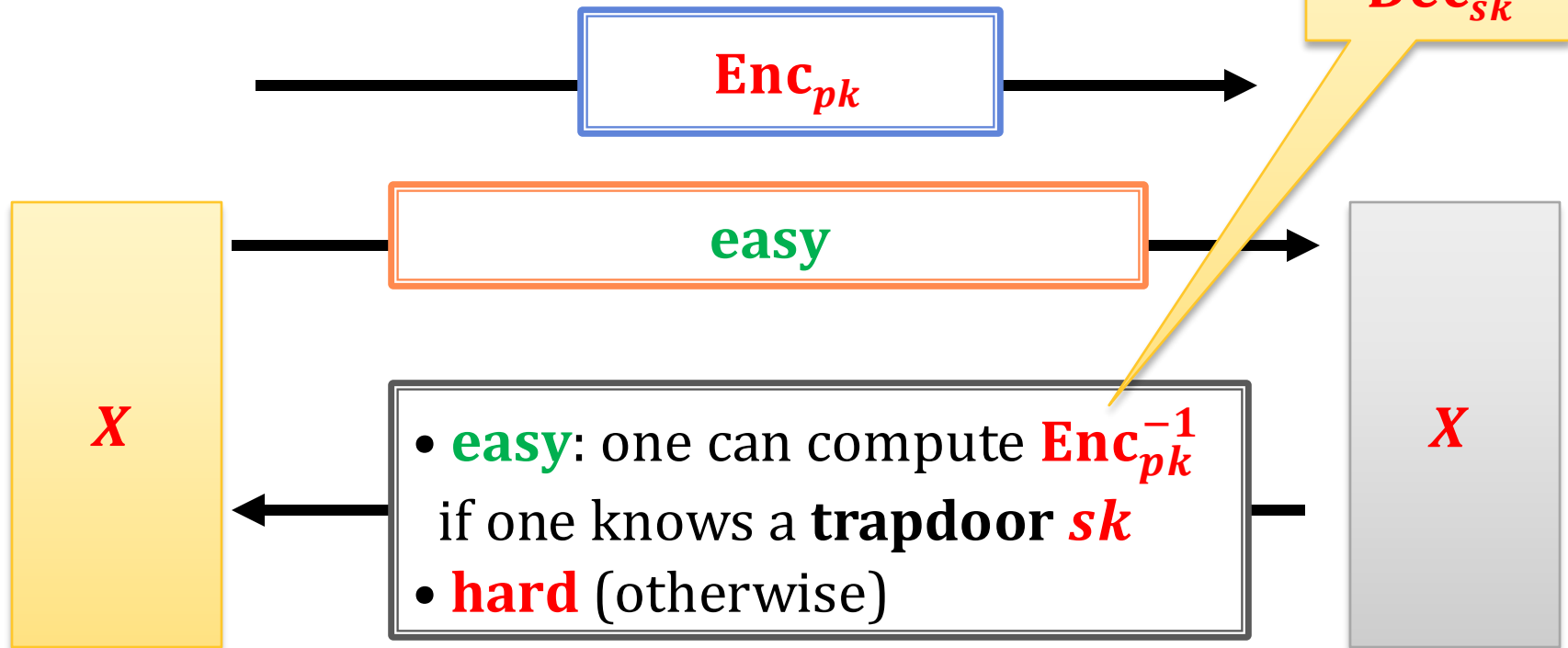
Trapdoor permutations (informal definition)

A family of permutations indexed by $pk \in \text{keys}$:

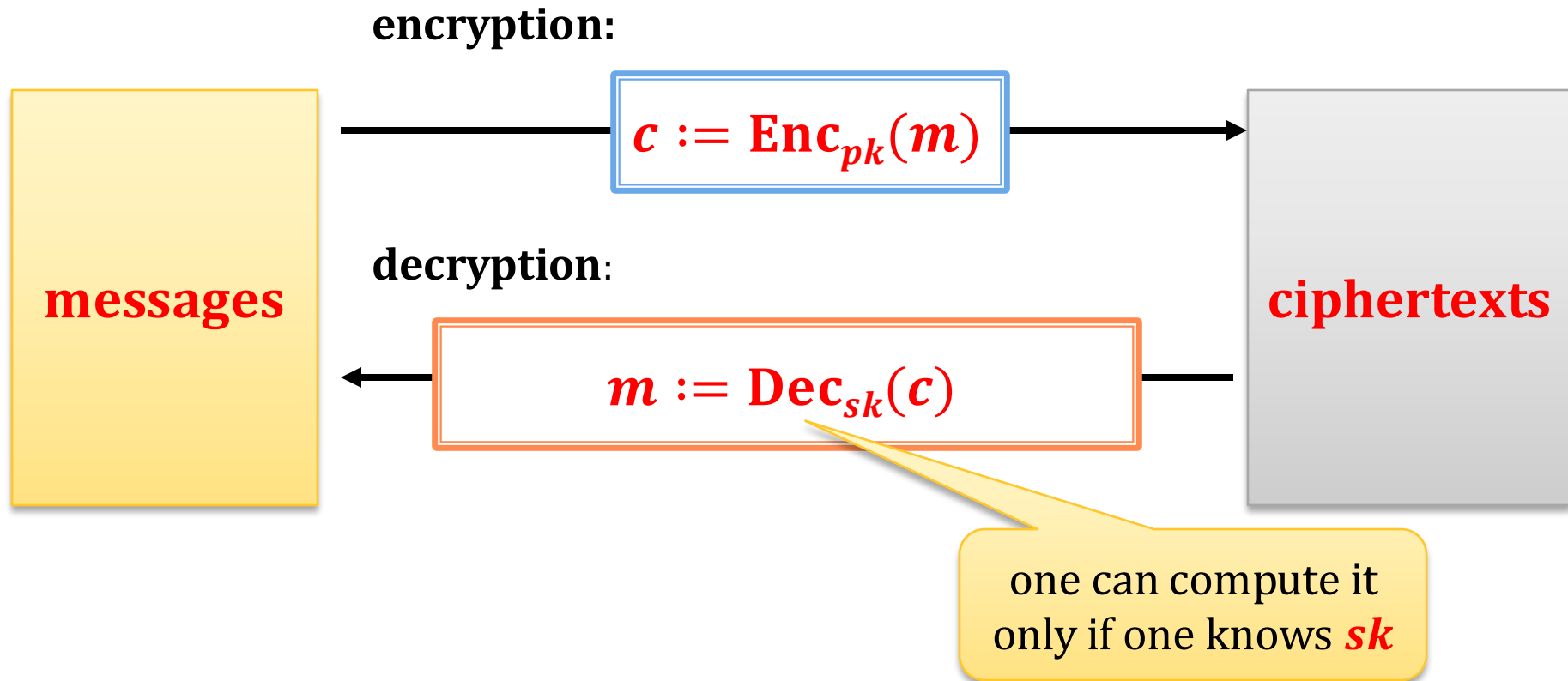
$$\{\text{Enc}_{pk} : X \rightarrow X\}_{pk \in \text{keys}}$$

such that for every key pk there exists a key sk , and:

this is
denoted
 Dec_{sk}

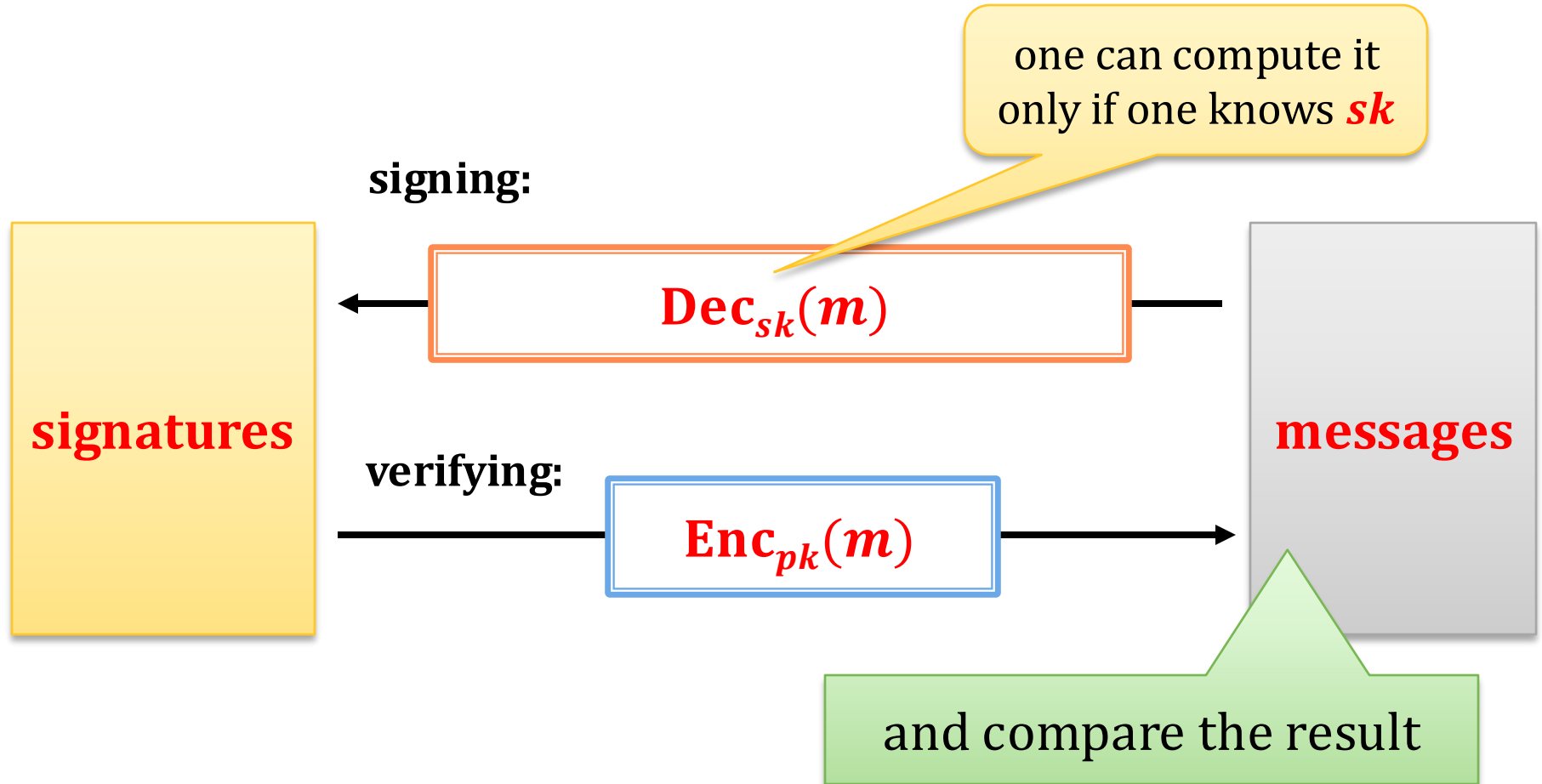


How to encrypt a message m



Warning: In reality it's not that simple. We will explain it later.

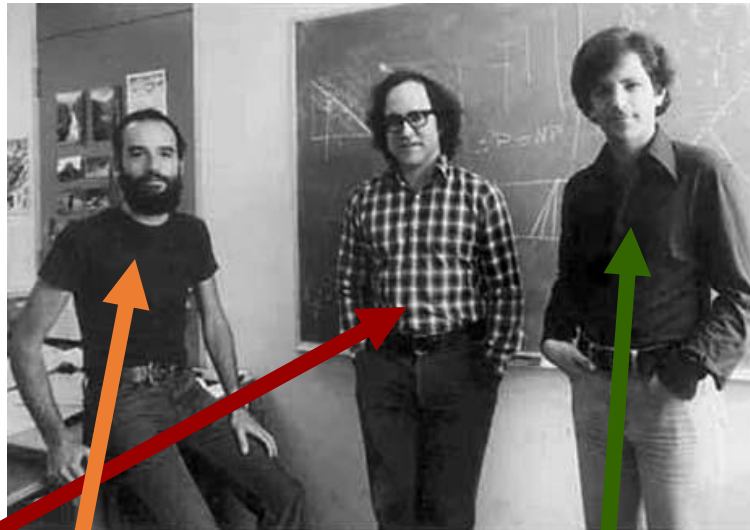
How to sign a message m



Warning: In reality it's not that simple. We will explain it later.

Do such functions exist?

Yes: exponentiation modulo N , where N is a product of two large primes.



Ron Rivest, Adi Shamir, and Leonard Adleman (1977)

RSA function is (conjectured to be) a trapdoor permutation!

The RSA function

$N = pq$, such that p and q are random primes,
and $|p| = |q|$

e – random such that $e \perp (p-1)(q-1)$

d – random such that $ed = 1 \pmod{(p-1)(q-1)}$

$pk := (N, e) \quad sk := (N, d)$

$\text{Enc}_{pk}: Z_N \rightarrow Z_N$ is defined as:

$$\text{Enc}_{pk}(m) = m^e \bmod N.$$

$\text{Dec}_{sk}: Z_N \rightarrow Z_N$ is defined as:

$$\text{Dec}_{sk}(c) = c^d \bmod N.$$

Questions and doubts

$N = pq$, such that p and q are random primes,
and $|p| = |q|$

e – random such that $e \perp (p-1)(q-1)$

d – random such that $ed = 1 \pmod{(p-1)(q-1)}$

$pk := (N, e)$ $sk := (N, d)$

$Enc_{pk}: Z_N \rightarrow Z_N$ is defined as:

$$Enc_{pk}(m) = m^e \bmod N.$$

$Dec_{sk}: Z_N \rightarrow Z_N$ is defined as:

$$Dec_{sk}(c) = c^d \bmod N.$$

encryption is
deterministic...

How **large** these
primes need to be?
How to **sample** them?

where does this come
from?

Can **exponentiation**
be done **efficiently**?

$Enc_{pk}(1) = 1^e \bmod N = 1$
Oops...

We will address them later...

(N, e, d) – as on the previous slide

“Handbook” RSA

Handbook RSA encryption scheme:

messages and ciphertexts: $\mathbb{Z}_N$

- $\text{Enc}_{N,e}(m) = m^e \bmod N$
- $\text{Dec}_{N,d}(c) = c^d \bmod N$

Handbook RSA signature scheme:

messages and signatures: $\mathbb{Z}_N$

- $\sigma := \text{Sign}_{N,d}(m) = m^d \bmod N$
- $\text{Vrfy}_{N,e}(m, \sigma) = \text{output yes iff } \sigma^e \bmod N = m$

Is **RSA** secure?

Is **RSA** secure:

1. as an **encryption scheme**?
2. as a **signature scheme**?

The answer is not that simple.

First, we would need to define security!

We will do it on the next lectures.

Symmetric vs asymmetric crypto

Symmetric cryptography (also called: private key cryptography) is **much more efficient!**

Example (Intel Core 2 1.83 GHz processor):

	MiB/Second	Cycles/Byte
AES/CTR (128-bit key)	139	12.6
HMAC(SHA-1)	147	11.9

	Operations/Second	Megacycles/Operation
RSA 2048 Encryption	6,250	0.29
RSA 2048 Signature	165	11.06

Source: <https://www.cryptopp.com/benchmarks.html>

Practical solutions

Typically **asymmetric cryptography** is **combined** with the **symmetric one**.

For example: asymmetric cryptography is used only for **agreeing on a symmetric key**.

Or: one can combine it directly using a “**hybrid approach**”.

(we will discuss it later)

Plan

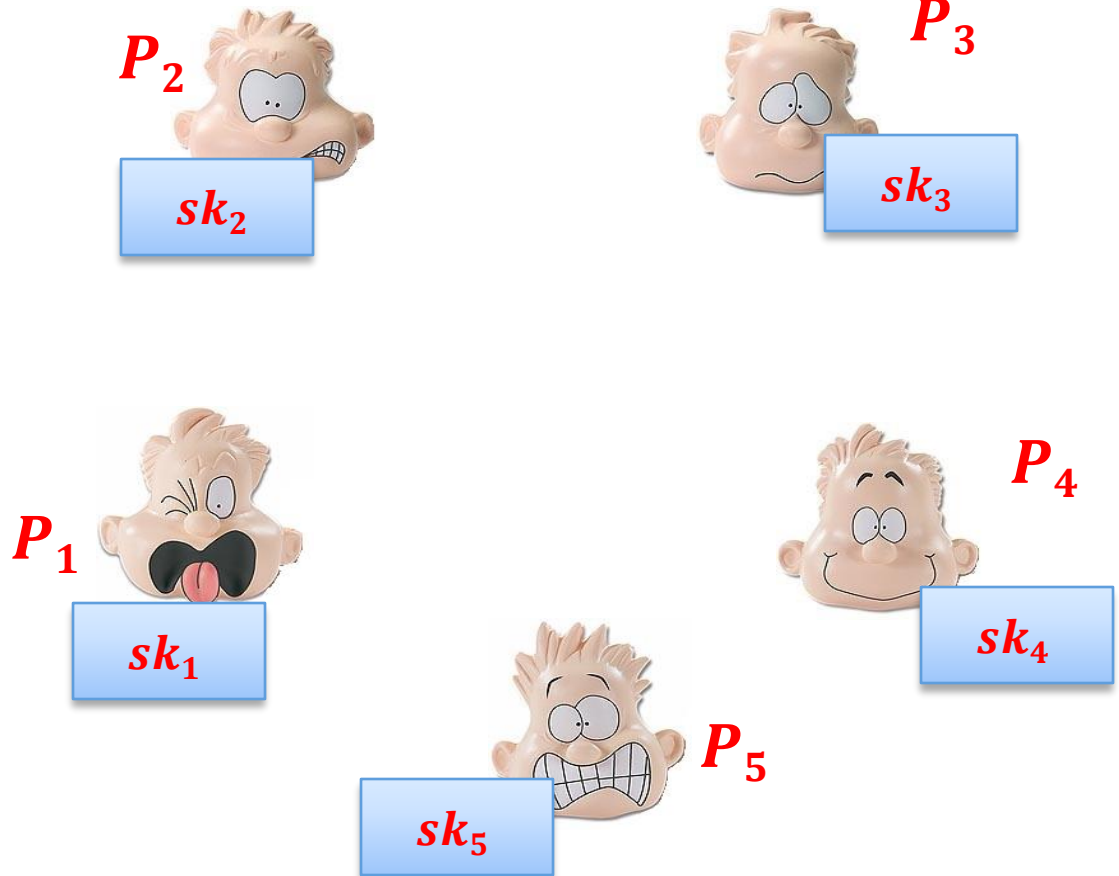
1. Public key cryptography – an overview
2. The key management problem
 1. qualified signatures
 2. public key infrastructure
3. Identity-based cryptography



Remember this slide?

public register:

pk_1
pk_2
pk_3
pk_4
pk_5

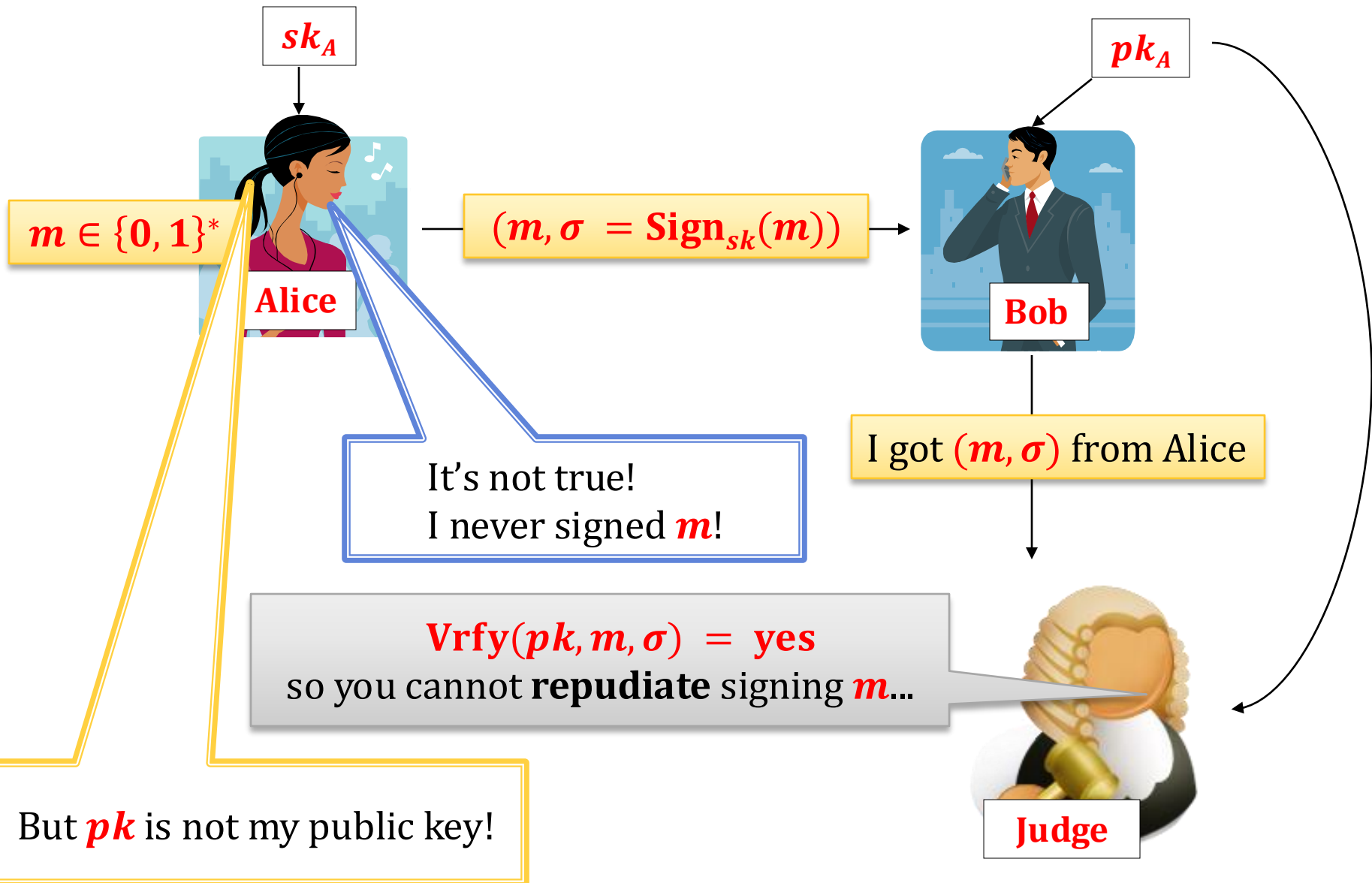


Question:

How to maintain the public register?

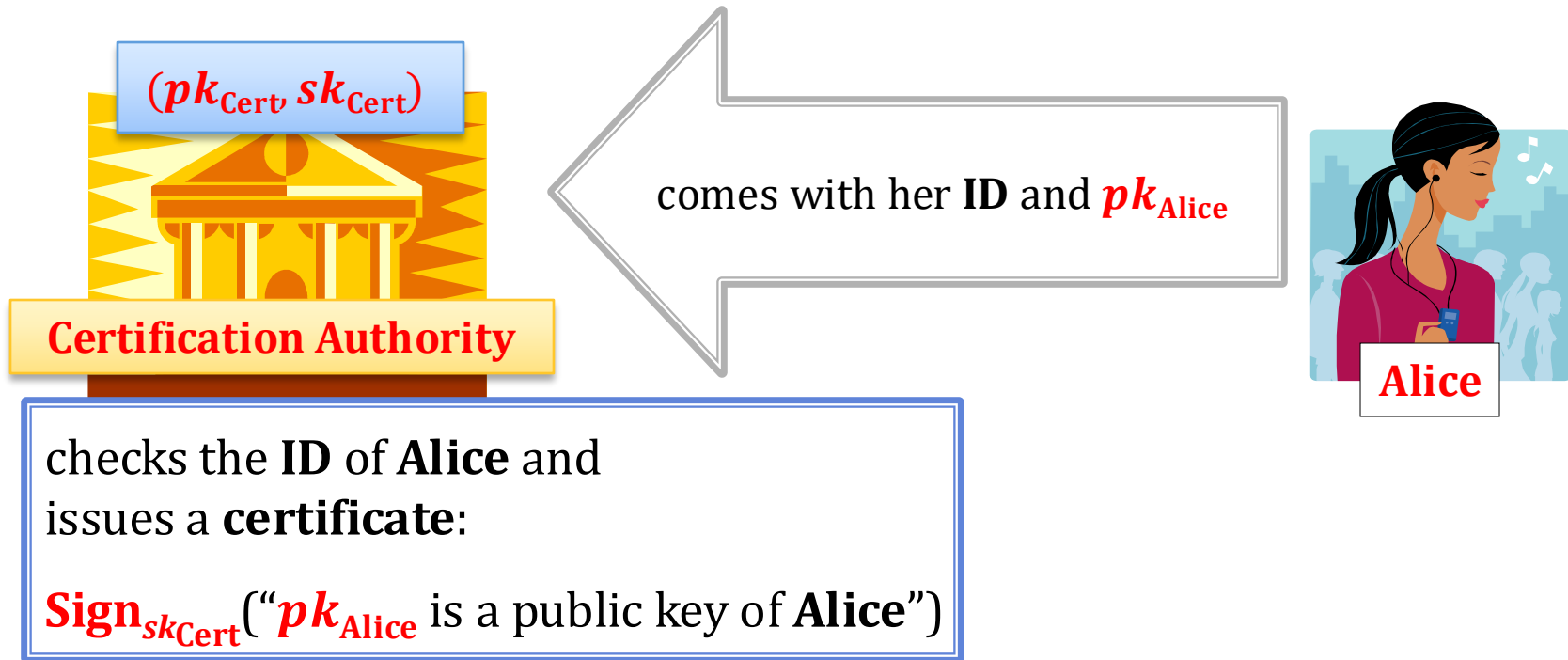
1. We start with the case when the public keys are used for **signing that is legally binding**.
2. Then we consider other cases.

A problem



Solution: certification authorities

A simplified view:



Now, **everyone** can verify that pk_{Alice} is a public key of **Alice**.
So **Alice** can attach it to every signature

really everyone?

What is needed to verify the certificate

To verify the certificate coming from **Cert** one needs:

1. to **know** the public key of the **Cert**
2. to **trust** **Cert**.

It is better if **Cert** also keeps a document:

*“I, **Alice** certify that **pk**_{Alice} is my public key”*
with a **written** signature of **Alice**.

How does it look from the legal point of view?

What matters at the end is if you can **convince the judge**.

Many countries have now a special law regulating these things.

In **Poland**:

[Ustawa o podpisie elektronicznym](#), z dnia 18 września 2001 r.

(Dz.U.01.130.1450) 28 str. ([ISIP](#)), na podst. dyrektywy EU [1999/93/EC](#)

This law defines the conditions to become an official **certification authority**.

A certificate issued by such an authority is called a **qualified certificate**.

A signature obtained this way is called a **qualified digital signature**.

The **qualified signature** is equivalent to the written one!

Polish Certificate Authorities:

NCCERT | NBP

ENGLISH | ARCHIWUM | KONTAKT

NBP

Narodowy Bank Polski

Narodowe Centrum Certyfikacji (NCCert)

Strona główna

Dokumenty

Podmioty kwalifikowane

Zaświadczenia certyfikacyjne

Lista CRL

Lista TSL

Komunikaty

2015

2013

2012

2011

2010

2009

2008

2007

2006

2005

REJESTR KWALIFIKOWANYCH PODMIOTÓW ŚWIADCZĄCYCH USŁUGI CERTYFIKACYJNE

Narodowy Bank Polski prowadzi rejestr podmiotów kwalifikowanych od dnia 1 października 2005r.

 Plik: [NCCert.crt](#) - zaświadczenie certyfikacyjne Narodowego Centrum Certyfikacji - (nowy root)

Wpisy uszeregowane pod kątem czasu uzyskania wpisu do rejestru - w kolejności od najwcześniejszego

Numer wpisu	Nazwa podmiotu	Rodzaj świadczonych usług	Czas dokonania wpisu
1.	UNIZETO TECHNOLOGIES Spółka Akcyjna	Wydawanie kwalifikowanych certyfikatów	31 grudnia 2002 r., godz. 12:00:00
		Wydawanie kwalifikowanych certyfikatów atrybutów	13 września 2007 r., godz. 10:00:00
		Znakowanie czasem	24 stycznia 2003 r., godz. 12:00:00

So, what to do if you want to issue the qualified signatures?

You have to go to one of these companies and **get a qualified certificate** (it costs!).

The certificate is **valid just for some period**.

What if the secret key is lost?

In this case you have to **revoke** the certificate.

Every authority maintains a list of **revoked certificates**.

The certificates come with some **insurance**.

In many case one doesn't want to use the qualified signatures

The certificates cost.

It's **risky** to use them:

How do you know what your computer is really signing?
Computers have **viruses, Trojan horses**, etc.

You can use **external trusted hardware** but it should have a display (so you can see what is signed).

Remember: qualified signatures are equivalent to the written ones!

Plan

1. Public key cryptography – an overview
2. The key management problem
 1. qualified signatures
 2. public key infrastructure
3. Identity-based cryptography



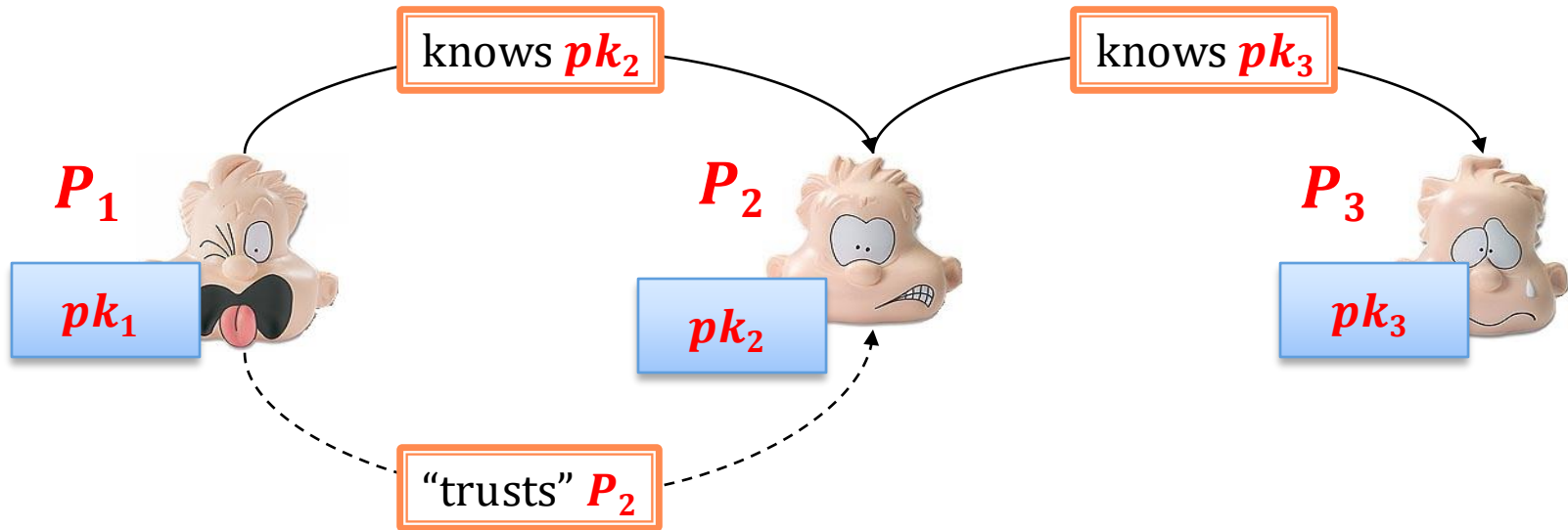
Practical solution

In many cases the **qualified signatures** are an overkill.

Instead, people use **non**-qualified signatures.

The certificates are distributed using a **public-key infrastructure (PKI)**.

Users can certify keys of the other users

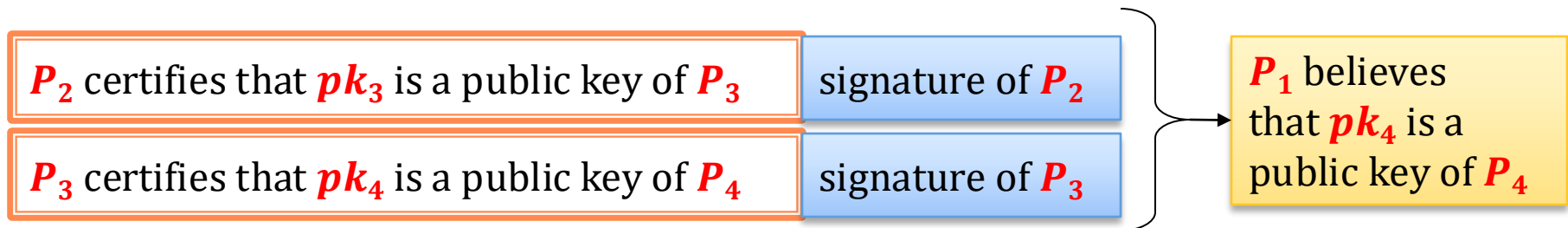
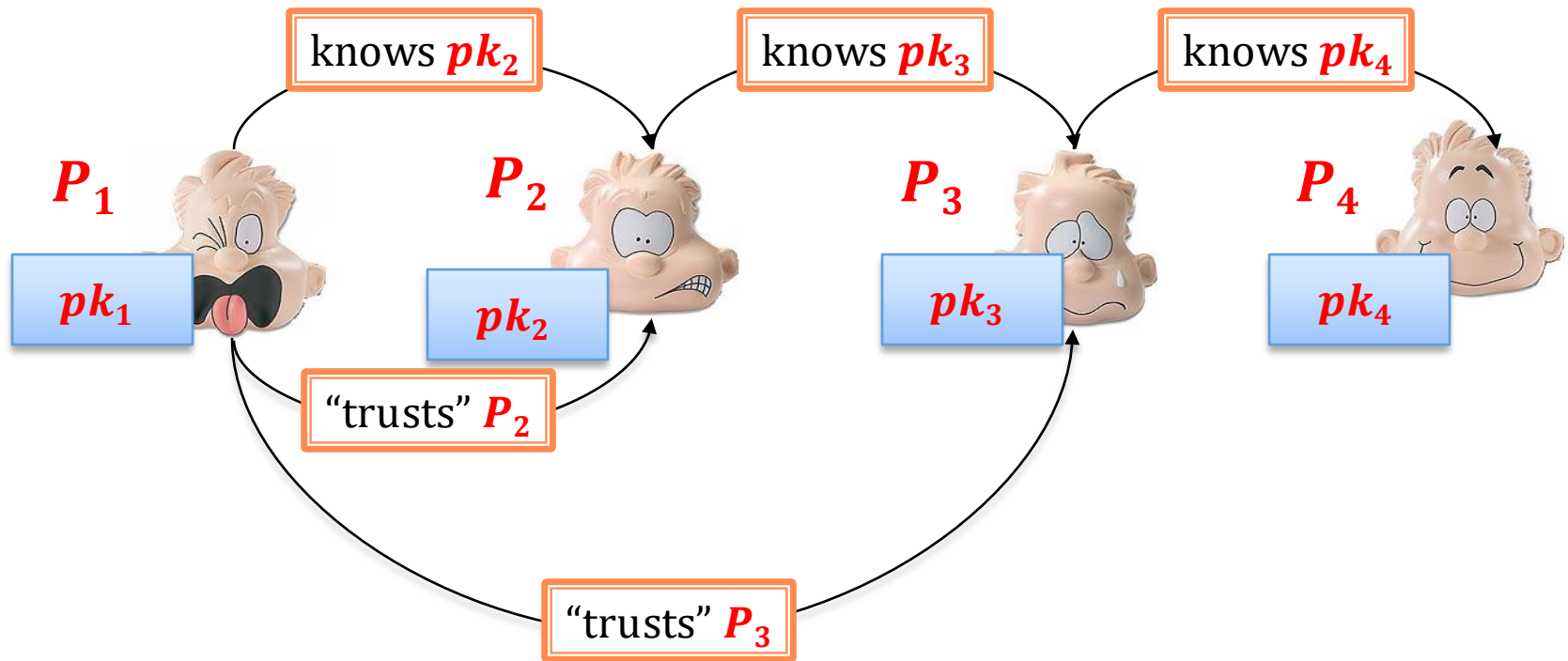


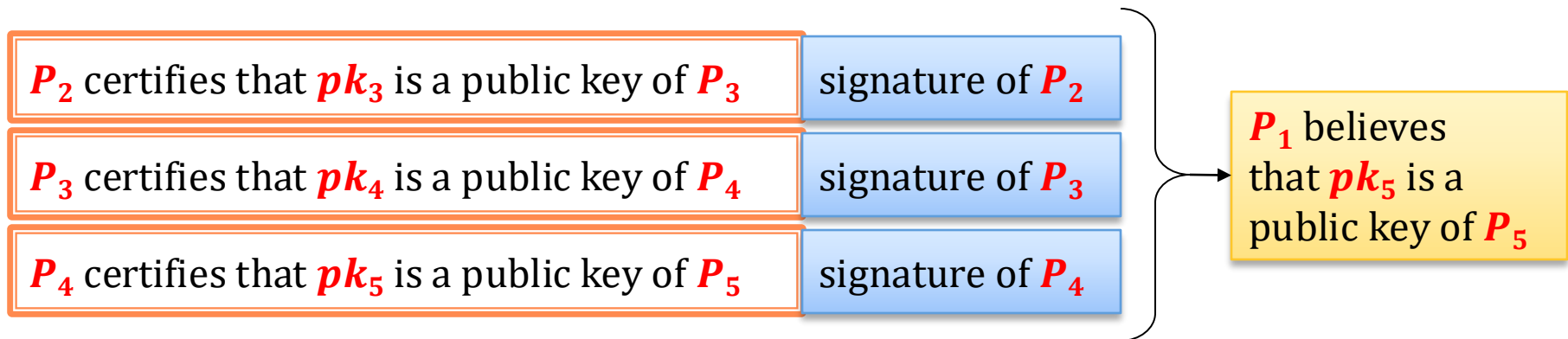
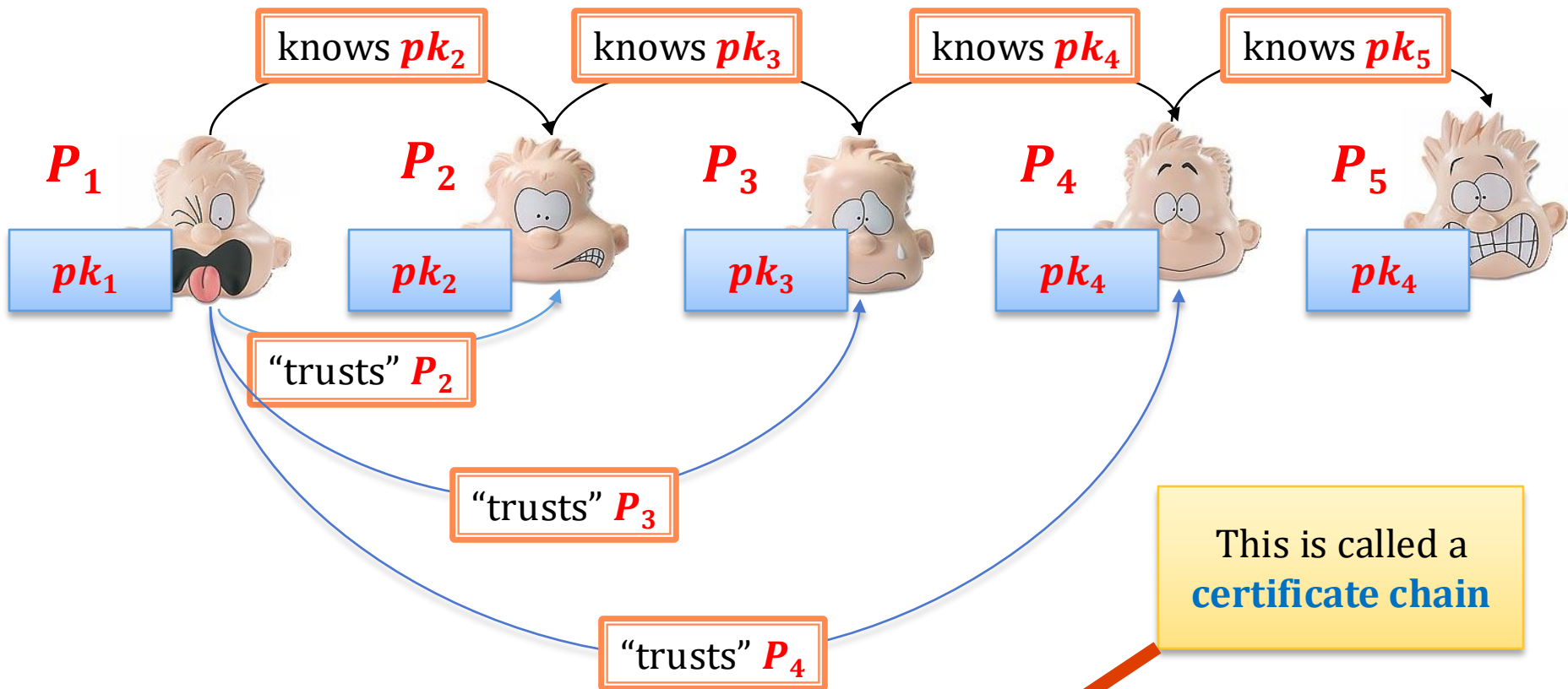
P_2 certifies that pk_3 is a public key of P_3

signature of P_2

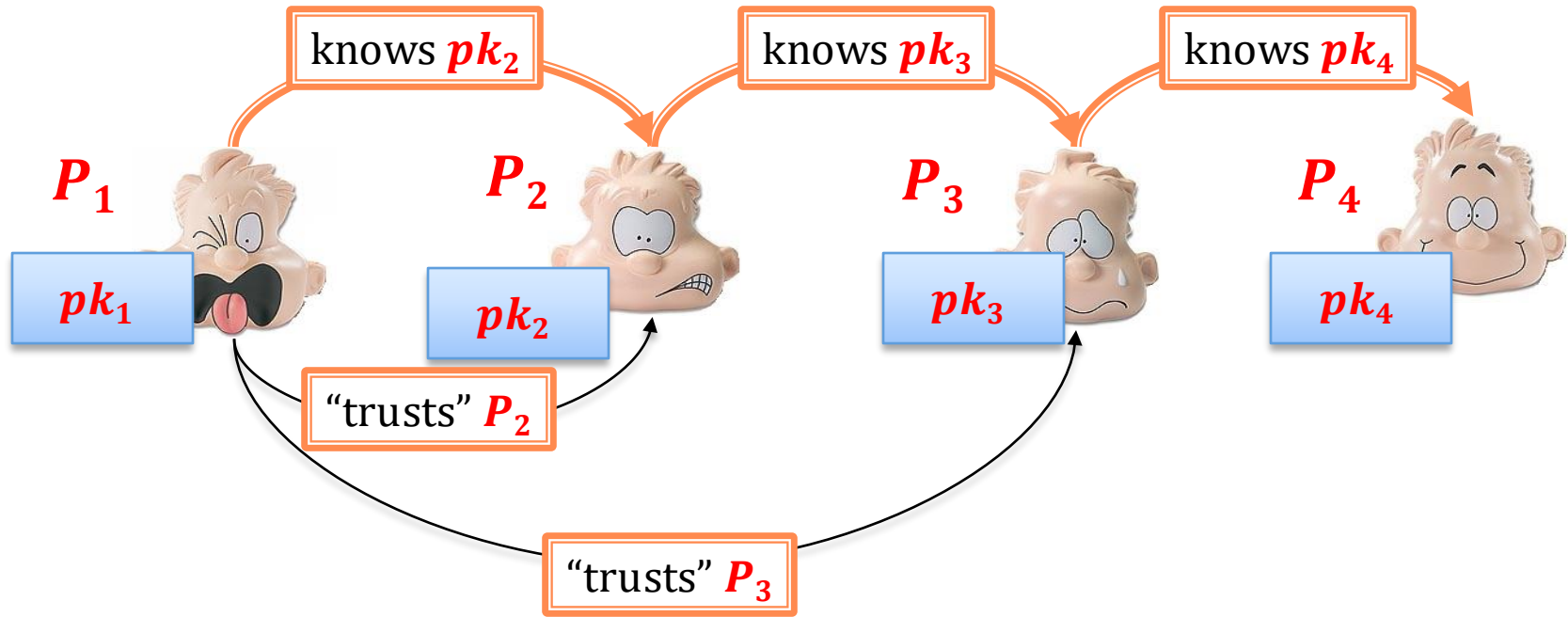
P_1 believes that pk_3 is a public key of P_3

this should be done only if P_2 really met P_3 in person and verified his identity





A problem



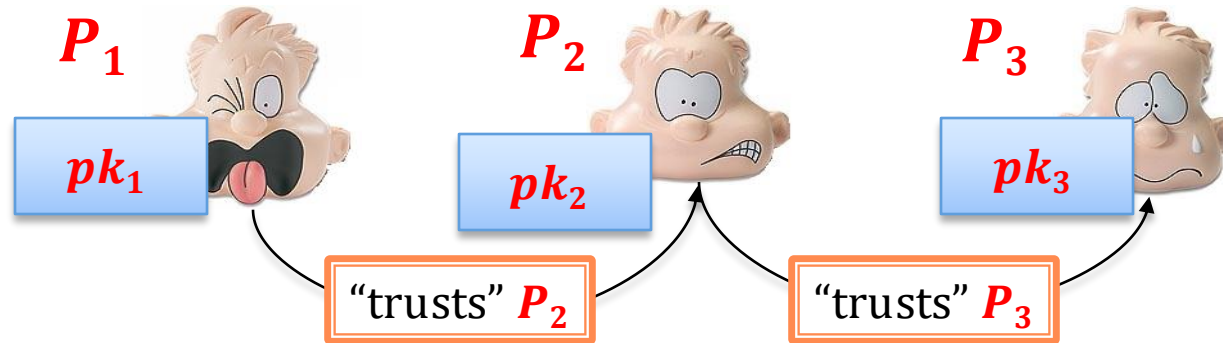
What if P_1 does not know P_3 ?

How can he trust him?

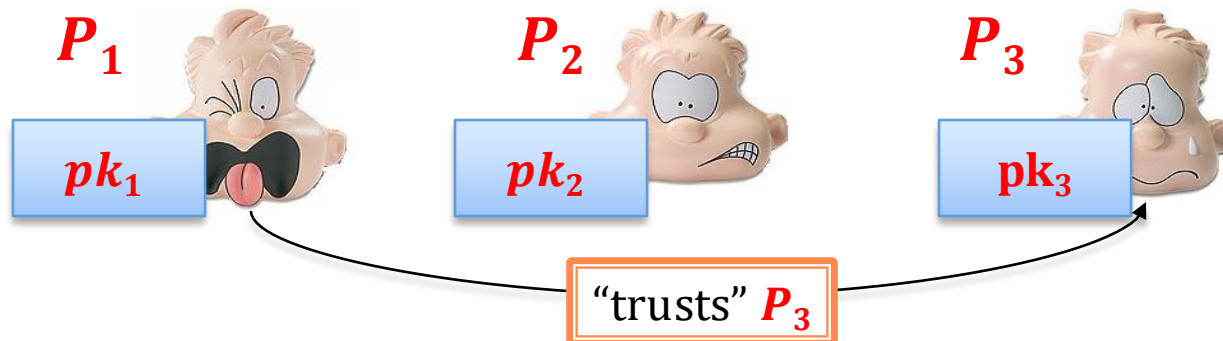
Answer: P_2 can recommend P_3 to P_1 .

A question: is trust transitive?

Does:

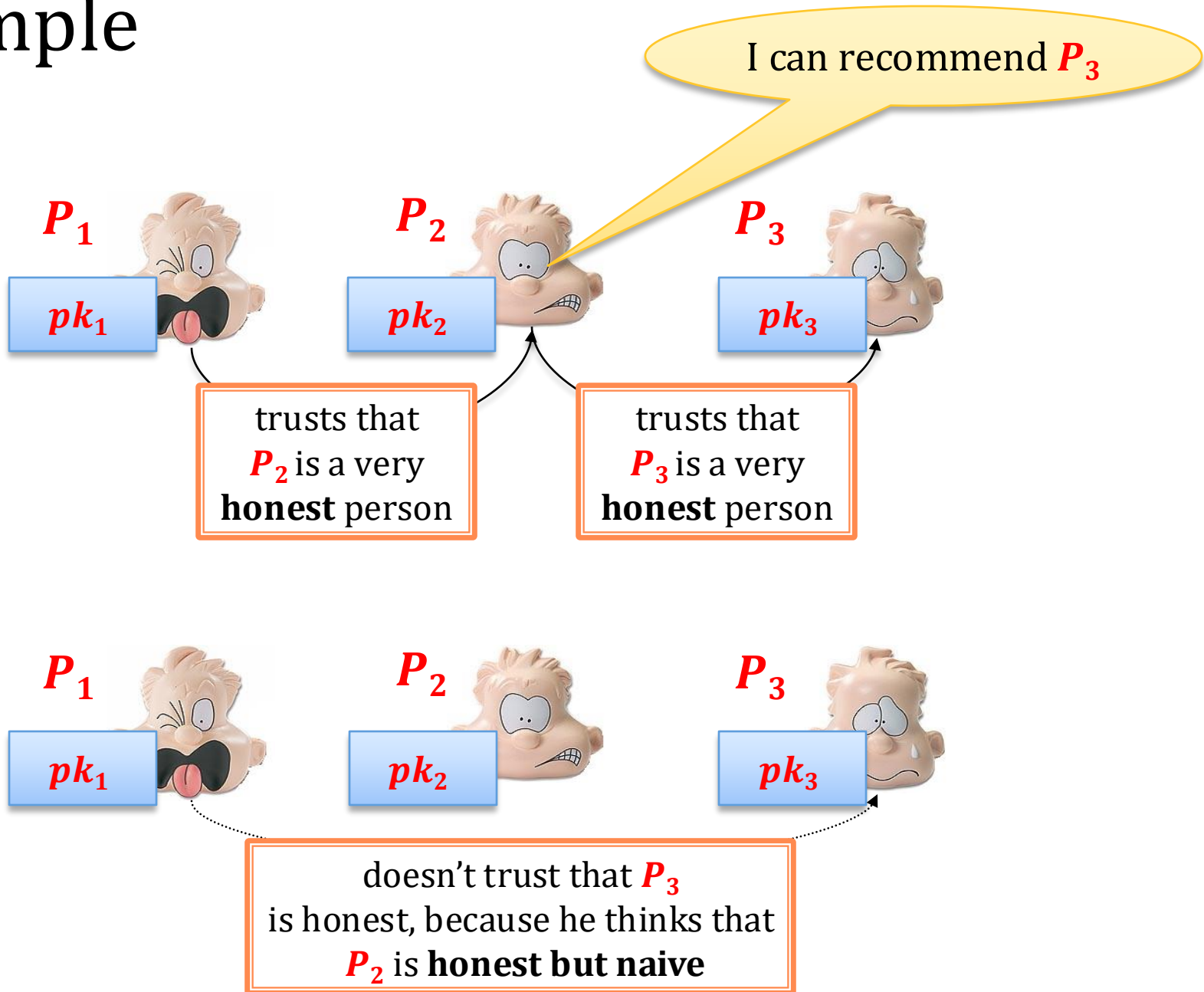


imply:



?

Example



Moral

Trust is not transitive:

“ P_1 trusts in the certificates issued by P_2 ”

is not the same as:

“ P_1 trusts that

if

P_2 says: “you can trust the certificates issued by P_3 ”

then

one can trust the certificates issued by P_3 ”

Recommendation levels

level **1** recommendation:

A: *"you can trusts in all the certificates issued by **B**"*

level **2** recommendation:

A: *"you can trust that **all the level 1** recommendations issued by **B**"*

level **3** recommendation:

A: *"you can trust that **all the level 2** recommendations issued by **B**"*

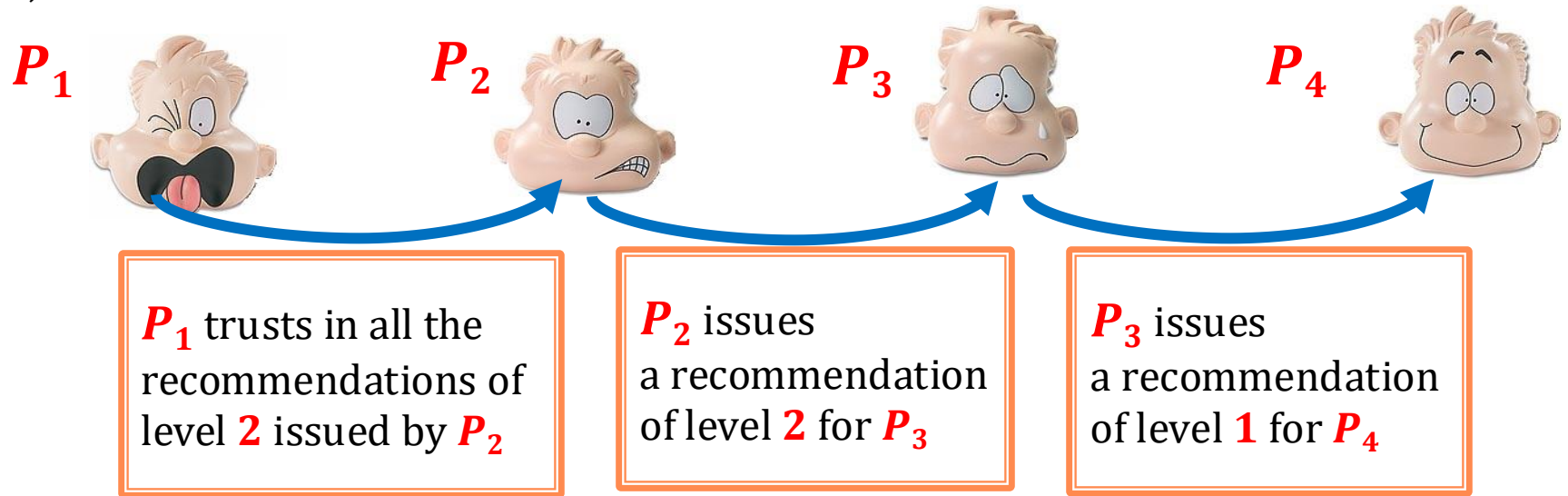
and so on...

Recursively:

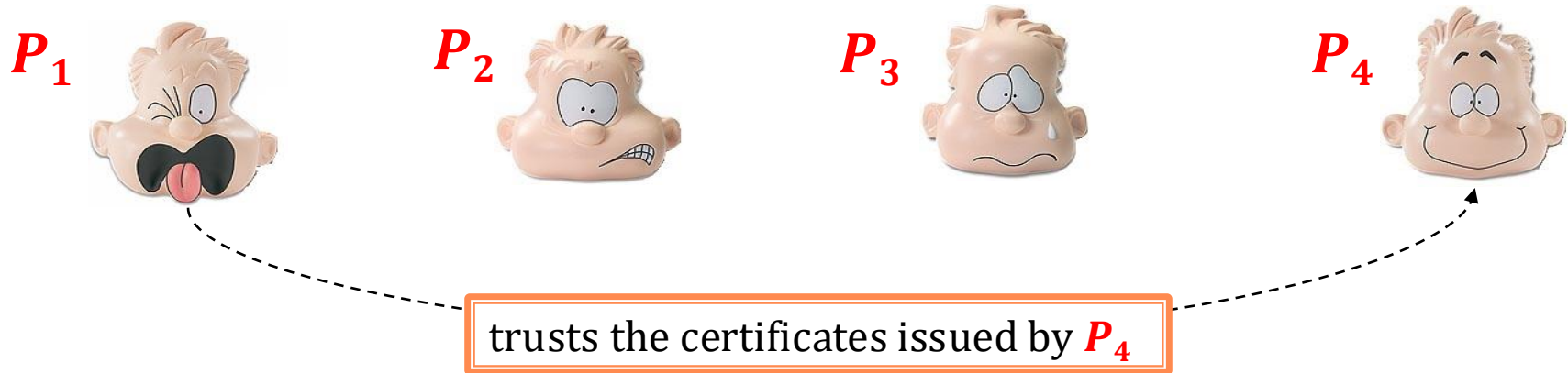
level **$i + 1$** recommendation:

A: *"you can trust that **all the level i** recommendations issued by **B**"*

Now, if:



then



Of course the recommendations also need to be signed.

Starts to look complicated...

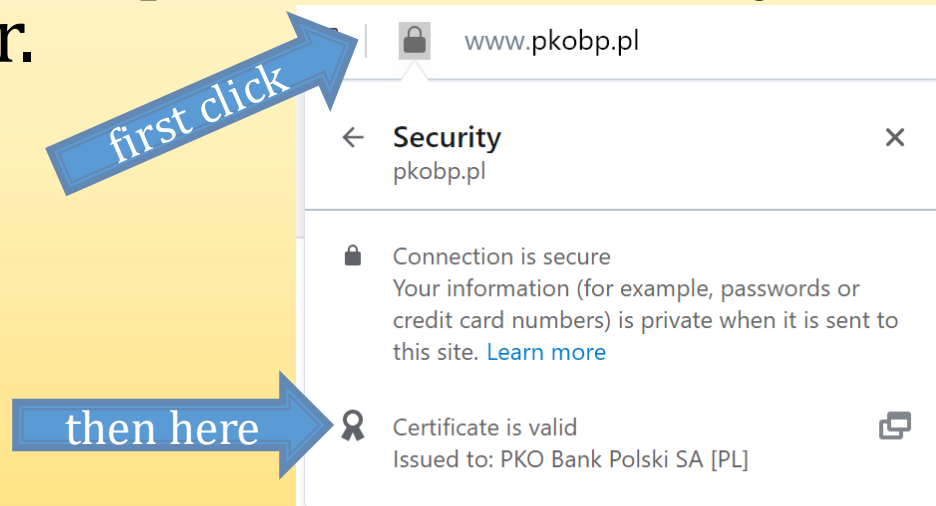
How is it solved in practice?

In popular standard is **X.509** the recommendation is included into a certificate.

Here the level of recommendations is bounded using a field called ***basic constraints***.

X.509 is used for example in **SSL**.

SSL is implemented is implemented in every popular web-browser.



The simplified picture

web browser knows these certificates

Verisign

DigiCert

Entrust

...

Implicit assumptions:

- the author of the browser is honest,
- the author of the browser is competent
- nobody manipulated the browser

is it
always
true?

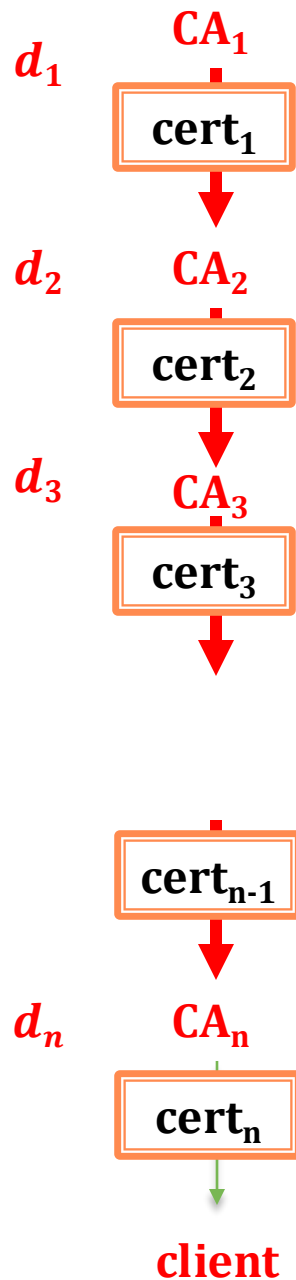
Verisign
Europe

Verisign
USA

Verisign
Italy

Banca di Roma

a certificate path



Each **cert_i** has a number **d_i** denoting a maximal depth of certificate chain from this point (this limits the recommendation depth)

That is, we need to have:

$$d_i \geq n - i$$

Other popular approaches

In the past: **Pretty Good Privacy** (PGP) – every user can act as a certification authority (hence the name: **Web of Trust**)

More recently: **Namecoin**– use blockchain as a distributed register.

Plan

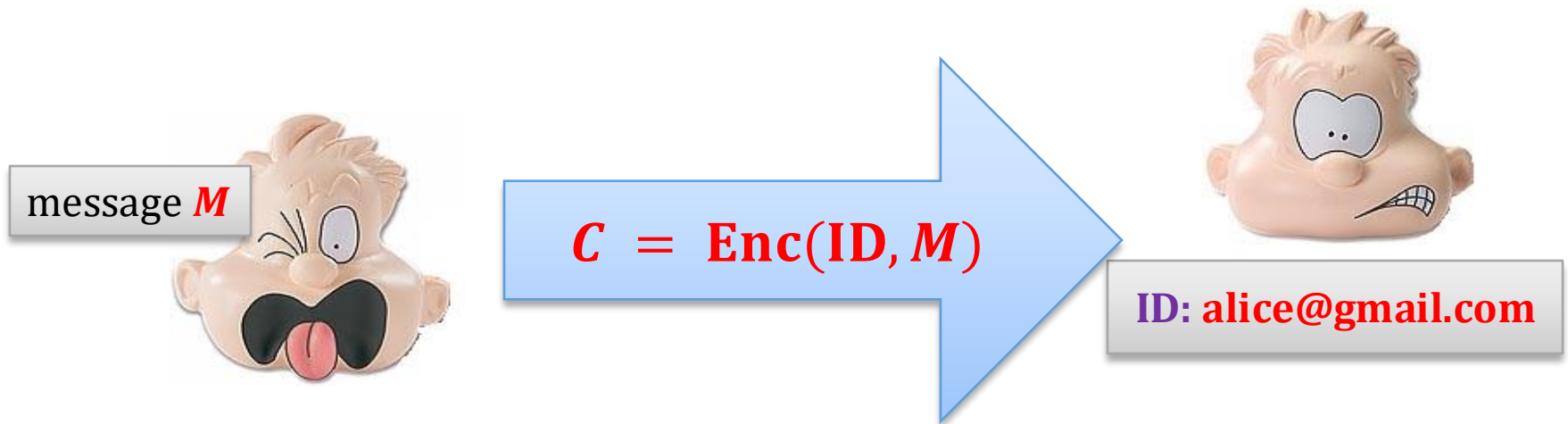
1. Public key cryptography – an overview
2. The key management problem
 1. qualified signatures
 2. public key infrastructure
3. Identity-based cryptography



Identity based cryptography

Main idea:

the identifier **ID** of the user is its public key.
(e.g. **ID** = user's email address).



question:

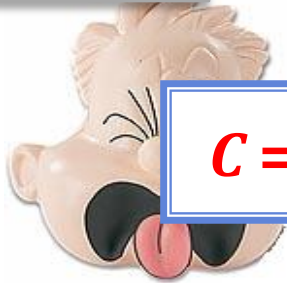
What is the private key?

Solution



How to decrypt

message M



$$C = \text{Enc}(\text{PK}, \text{alice@gmail.com}, M)$$



alice@gmail.com

knows

$SK_{\text{alice@gmail.com}}$

calculates

$$M = \text{Dec}(SK_{\text{alice}}, C)$$

ID-based encryption

Main **advantage**:

- no need for an “infrastructure”

Drawbacks:

- users need to **trust an authority**,
- and they need to have a **secure link** to it,
- what about the **key revocation**?

ID-based encryption

Proposed by **Adi Shamir** in **1984**.

(he only implemented the identity-based **signatures**)

First schemes were proposed by **Boneh** and **Franklin** (2001) and, independently **Cocks** (2001).

In **2002 Boneh** started a company

Voltage Security

that produces solutions based on his ID-based scheme.

©2025 by Stefan Dziembowski. Permission to make digital or hard copies of part or all of this material is currently granted without fee *provided that copies are made only for personal or classroom use, are not distributed for profit or commercial advantage, and that new copies bear this notice and the full citation.*